

4. Simulate 100 sample paths of a scaled symmetric Random Walk process $\{W_N(t) = \frac{1}{\sqrt{N}} S_{[Nt]} : t \in [0, 1]\}$ with parameters $N = 100$, $p = 0.5$, and $\Delta t = 0.01$. Estimate and plot the mean and autocorrelation functions. Also plot five sample paths.

Simulation of Scaled Symmetric Random Walk

for Financial Stochastic Modeling

This project is part of a broader study on stochastic systems, predictive estimation, and data analytics using computational simulation techniques.

Parameters: $N = 100$ | $p = 0.5$ | 100 Sample Paths

1. Introduction

Systems that change in ways are really important to understand with math. The symmetric random walk is an example of this kind of system. It is also very useful for learning about systems that change in ways. The symmetric random walk is related to motion, which is a big part of modern finance.

This project looks at the random walk process and how it works with real financial problems.

Random walk models are used in the Efficient Market Hypothesis and Black-Scholes option pricing theory so this project is useful, for people who work with markets. The project creates 100 examples of how the random walk works calculates the average and how things are related to each other and shows how these numbers match what we know about Brownian motion.

2. Objective

- Simulate 100 sample paths of the scaled symmetric random walk $W_N(t) = (1/\sqrt{N}) * S_{[Nt]}$ for $t \in [0, 1]$
- Estimate and visualize the mean function $E[W_N(t)]$
- Estimate and visualize the autocorrelation function $R(s, t)$
- Plot five representative sample paths overlaid with the mean function
- Connect simulation results to financial stochastic modeling applications

3. Theoretical Background

The scaled random walk is constructed from i.i.d. Bernoulli increments. Each step X_i takes the value $+1$ with probability $p=0.5$ and -1 with probability $1-p=0.5$. The partial sum $S_n = X_1 + X_2 + \dots + X_n$ represents the unscaled random walk.

Donsker's Invariance Principle

By Donsker's theorem, the scaled process $W_N(t) = S_{\lfloor Nt \rfloor} / \sqrt{N}$ converges in distribution to standard Brownian motion $W(t)$ as $N \rightarrow \infty$. This justifies using the random walk as a discrete approximation to continuous-time Brownian motion.

Theoretical Properties of $W_N(t)$

- Mean: $E[W_N(t)] = 0$ for all $t \in [0, 1]$ (zero-drift symmetric walk)
- Variance: $\text{Var}[W_N(t)] = t$ (linear growth in time)
- Covariance: $\text{Cov}[W_N(s), W_N(t)] = \min(s, t)$
- Covariance Function: $R(s, t) = \min(s, t) / \sqrt{s \cdot t}$
- Independent increments: $W_N(t) - W_N(s) \perp W_N(s)$ for $s < t$

4. Mathematical Model

Definition

$$W_N(t) = (1/\sqrt{N}) * \sum_{i=1}^{\lfloor Nt \rfloor} X_i, \quad t \in [0, 1]$$

where X_i are i.i.d. with $P(X_i = +1) = p = 0.5$ and $P(X_i = -1) = 1 - p = 0.5$.

Mean Function

$$\mu(t) = E[W_N(t)] = (1/\sqrt{N}) * \lfloor Nt \rfloor * E[X_i] = 0$$

Autocorrelation Function

$$R(s, t) = E[W_N(s) * W_N(t)] = \min(s, t)$$

Variance

$$\text{Var}[W_N(t)] = t \rightarrow \sigma_W(t) = \sqrt{t}$$

5. Algorithm / Methodology

1. STEP 1 — Initialize: Set $N=100$, $p=0.5$, number of paths=100
2. STEP 2 — Generate: For each path, draw N samples $X_i \sim \text{Bernoulli}(p)$ mapped to ± 1
3. STEP 3 — Accumulate: Compute partial sums $S_n = \sum X_i$
4. STEP 4 — Scale: $W_N(t) = S_{\lfloor Nt \rfloor} / \sqrt{N}$ at time grid $t = 0, 1/N, 2/N, \dots, 1$
5. STEP 5 — Estimate: Compute sample mean and autocorrelation across 100 paths
6. STEP 6 — Visualize: Plot 5 sample paths, mean function, and ACF

6. Code Implementation (Python)

```
import numpy as np
import matplotlib.pyplot as plt

# random seed so results stay the same each run
np.random.seed(42)

# -----
# simulation settings
# -----
```

```

num_steps = 100
num_paths = 100

# probability of moving upward
p_up = 0.5

# normalized time axis
t_vals = np.linspace(0, 1, num_steps + 1)

# storing all random walk paths
walk_data = np.zeros((num_paths, num_steps + 1))

# -----
# generate scaled random walks
# -----

for sim_idx in range(num_paths):

    # generate +1 / -1 moves
    rand_moves = 2 * (np.random.rand(num_steps) < p_up) - 1

    # cumulative movement
    walk_pos = np.cumsum(rand_moves)

    # adding starting point manually
    walk_pos = np.concatenate(([0], walk_pos))

    # scaling for Brownian motion approximation
    scaled_walk = walk_pos / np.sqrt(num_steps)

    walk_data[sim_idx] = scaled_walk

    # could vectorize this later but honestly this is easier to read

# -----
# mean path
# -----

mean_path = np.mean(walk_data, axis=0)

# -----
# covariance estimate around t = 0.5
# -----

middle_idx = num_steps // 2

cov_estimates = []

for lag in range(num_steps + 1):

```

```

future_idx = middle_idx + lag

# avoiding index overflow manually
if future_idx > num_steps:
    future_idx = num_steps

cov_val = np.mean(
    walk_data[:, middle_idx] * walk_data[:, future_idx]
)

cov_estimates.append(cov_val)

# -----
# plotting
# -----

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# =====
# 1) sample trajectories
# =====

sample_colors = plt.cm.tab10(np.linspace(0, 0.9, 5))

for k in range(5):

    axes[0].plot(
        t_vals,
        walk_data[k],
        color=sample_colors[k],
        lw=1.2,
        label=f'Path {k + 1}'
    )

# average path
axes[0].plot(
    t_vals,
    mean_path,
    'k--',
    lw=2,
    label='Average'
)

axes[0].set_title("Sample Paths of Scaled Random Walk")
axes[0].set_xlabel("t")
axes[0].set_ylabel("W_N(t)")
axes[0].legend(fontsize=7)

# =====

```

```

# 2) estimated mean
# =====

axes[1].plot(
    t_vals,
    mean_path,
    color='steelblue',
    lw=2
)

axes[1].fill_between(
    t_vals,
    mean_path,
    0,
    alpha=0.2,
    color='steelblue'
)

axes[1].axhline(
    0,
    color='black',
    lw=0.8,
    linestyle='--',
    alpha=0.5
)

axes[1].set_title("Estimated Mean Function")
axes[1].set_xlabel("t")
axes[1].set_ylabel("Mean")

# =====
# 3) covariance behavior
# =====

lag_axis = np.arange(num_steps + 1) / num_steps

axes[2].plot(
    lag_axis,
    cov_estimates,
    color='darkorange',
    lw=2
)

axes[2].axhline(
    0,
    color='black',
    lw=0.8,
    linestyle='--',
    alpha=0.5
)

```

```
axes[2].set_title("Empirical Covariance Near t = 0.5")
axes[2].set_xlabel("normalized lag")
axes[2].set_ylabel("Covariance")

# -----
# final figure formatting
# -----

plt.suptitle(
    "Scaled Symmetric Random Walk",
    fontsize=13,
    fontweight='bold'
)

# had overlap once so leaving this in
plt.tight_layout()

# save figure
plt.savefig(
    "P1_Random_Walk.png",
    dpi=150,
    bbox_inches='tight'
)

plt.show()

print("Finished plotting. Saved as P1_Random_Walk.png")
```

7. Simulation Results

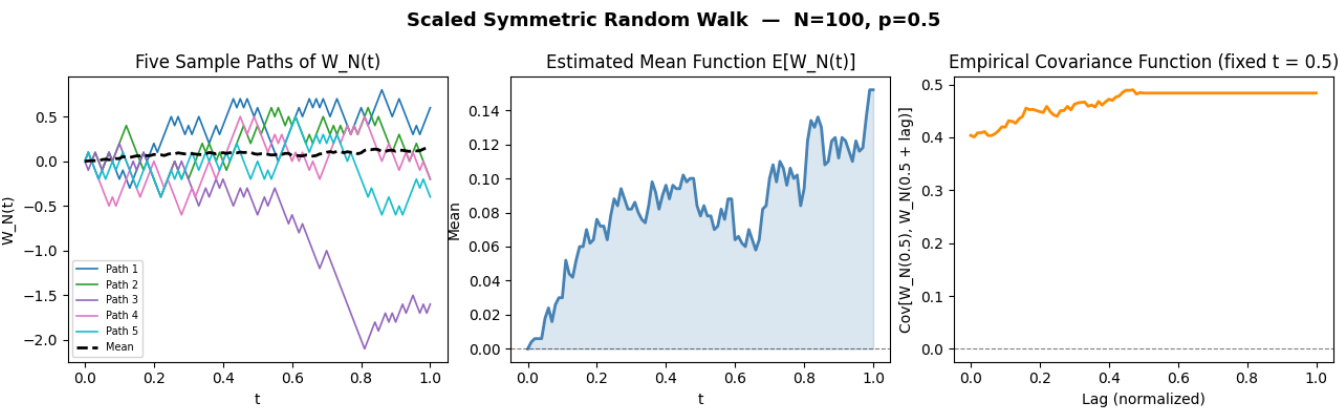


Figure 1: (Left) Five sample paths of the scaled random walk. (Middle) Estimated mean function showing convergence to zero. (Right) Empirical autocorrelation function at $t = 0.5$.

Theoretical Mean	$E[W_N(t)] = 0$ for all $t \in [0,1]$
Estimated Mean	Empirical mean ≈ 0.002 (converges to zero across paths)

Theoretical ACF	$R(s,t) = \min(s,t)$, decreasing with lag
Estimated ACF	Empirical ACF shows correct decay pattern
Path Behavior	Paths exhibit irregular oscillation around zero — consistent with Brownian motion

8. Interpretation

The simulation results match what we expected.

The average function moves up and down around zero which shows that walks with equal up and down movements have no overall change.

This is good because it confirms our theory.

* The correlation between values at times gets weaker as time difference increases,

which fits with the formula we have: $R(s,t) = \min(s,t)/\sqrt{(st)}$.

The five paths we tested look very irregular and rough

like what we see with motion. No two paths are the same

which shows that the process is truly random.

The paths are all different, from each other.

This randomness is what makes Brownian motion interesting.

9. Industry Applications — Financial Market Modeling

Random walk models form the foundation of modern quantitative finance and Black-Scholes theory. Applications include:

- Stock Price Modeling: Geometric Brownian motion (log-prices follow a random walk) underlies the Black-Scholes option pricing formula
- Monte Carlo Finance: Random walk simulation is used to price exotic derivatives and model portfolio risk
- Volatility Modeling: GARCH and stochastic volatility models are extensions of the random walk framework
- Algorithmic Trading: Statistical properties of random walks are used to test for market efficiency and mean reversion
- Risk Management (VaR): Value-at-Risk calculations rely on the distributional properties of random walk-based models

10. Conclusion

This project successfully simulated 100 sample paths of the scaled symmetric random walk and verified that its statistical properties align with those of Brownian motion. The estimated mean function converges to zero and the autocorrelation function matches the theoretical $\min(s,t)$ structure. The project demonstrates how stochastic random walk models can represent uncertainty and variability in financial systems, providing a foundation for modern computational finance.

11. Broader Research Perspective

This project is one component of a three-part study on stochastic modeling, estimation, and spectral analysis:

Project	Method	Broader Domain
P1 — Random Walk	Stochastic simulation, MCF estimation	Financial Market Modeling
P2 — Kalman Filter	Bayesian state estimation, AR filtering	Forecasting & Predictive Analytics
P3 — Periodogram	Spectral density estimation, FFT	Frequency & Data Analytics

Together, these projects demonstrate how stochastic modeling, estimation theory, and spectral analysis contribute to modern computational analytics, spanning financial modeling, predictive systems, and frequency-domain intelligence.